

Sistema de Vigilância: Alocação Heurística de Câmeras em Pontos Cegos/Críticos

Uriel Pacheco de Souza¹, Emanuel Scharmitzel Johanson¹, Kelwin Efrain Bagnhuk da Silva¹

¹Ciência da Computação – Universidade do Estado de Santa Catarina
Joinville – Santa Catarina – Brasil

{uriel.souza, emanuel.sj23, kelwin.silva}@edu.udesc.br

Abstract. *This paper presents a heuristic algorithm for camera placement in surveillance systems, addressing a variation of the classic Art Gallery Problem, which is NP-hard. The proposed solution incorporates two real-world constraints often ignored in simplified versions of the problem: obstacle-aware line-of-sight (via raycasting) and limited camera field-of-view (FOV) with fixed direction. A greedy set-cover heuristic combined with a local search refinement step is implemented in C and evaluated on synthetic warehouse and bank-agency floor plans, achieving coverage rates above 74% and 94% respectively within polynomial time.*

Resumo. *Este artigo apresenta um algoritmo heurístico para posicionamento de câmeras em sistemas de vigilância, tratando uma variação do clássico Problema da Galeria de Arte, que é NP-difícil. A solução incorpora duas restrições do mundo real: linha de visão sensível a obstáculos (via raycasting) e campo de visão (FOV) limitado com direção fixa. Uma heurística gulosa de cobertura de conjuntos, combinada com busca local, foi implementada em C e avaliada em plantas sintéticas de um galpão e de uma agência bancária, atingindo taxas de cobertura de 74% e 94%, respectivamente, em tempo polinomial.*

1. Introdução

Sistemas de vigilância por câmeras são amplamente utilizados na proteção de galpões, bancos e demais ambientes que exigem monitoramento contínuo. Um problema recorrente é decidir **onde** instalar as câmeras e **para onde** apontá-las de modo a cobrir a maior área possível, minimizando *pontos cegos* — regiões que nenhuma câmera enxerga.

Esse problema é uma variação do **Problema da Galeria de Arte** (*Art Gallery Problem*), formulado por Klee em 1973 e classificado como NP-difícil [2]. Discretizado sobre uma grade, ele se aproxima do problema de **Cobertura de Conjuntos** (*Set Cover*), também NP-difícil [3], no qual cada câmera candidata define um subconjunto de células que cobre, e deseja-se selecionar o menor número de subconjuntos cuja união cubra todo o espaço de interesse. Uma visão geral do problema de vigilância e de suas variantes práticas pode ser encontrada em [1].

Diferentemente de formulações idealizadas (câmeras 360° sem obstáculos, ou cobertura de sinal Wi-Fi que atravessa paredes com atenuação), uma câmera real jamais enxerga através de paredes e possui ângulo de abertura fixo, não giratório em tempo real. Este trabalho propõe uma modelagem que respeita essas restrições e resolve o problema

por meio de uma heurística construtiva gulosa seguida de busca local, implementadas em C.

O artigo está organizado assim: a Seção 2 define o problema e as variáveis reais incorporadas; a Seção 3 descreve a solução heurística; a Seção 4 apresenta os testes e resultados; e a Seção 5 conclui o trabalho.

2. Definição do Problema

2.1. Modelagem e NP-dificuldade

O ambiente é representado por uma matriz M de dimensões $L \times C$, em que $M[i][j] = 0$ indica célula livre e $M[i][j] = 1$ indica parede ou obstáculo, que bloqueia a visão de qualquer câmera. Uma *câmera candidata* é o par (p, d) , com p célula livre e $d \in \{N, S, L, O\}$ a direção fixa da câmera. O objetivo é selecionar até K câmeras candidatas maximizando o número de células cobertas.

Este problema não possui algoritmo de tempo polinomial conhecido: (i) é uma variação direta do Problema da Galeria de Arte, NP-difícil mesmo em sua versão mais simples [2]; (ii) corresponde a uma instância do problema de Cobertura de Conjuntos, cuja resolução exata também é NP-difícil [3]; (iii) uma busca exaustiva avaliaria todos os subconjuntos de câmeras candidatas — com N células livres e 4 direções, existem $4N$ candidatas e $O(2^{4N})$ subconjuntos, um crescimento exponencial que torna a solução exata inviável (um galpão de 148 células livres geraria mais de 2^{592} combinações), consistente com a caracterização de problemas NP-difíceis como aqueles cuja solução exata exige tempo ou memória proibitivos [4].

2.2. Variáveis do mundo real incorporadas

Duas restrições, tipicamente ignoradas em formulações simplificadas, foram modeladas explicitamente:

- (A) **Linha de visão obstruída:** uma câmera não enxerga através de paredes. Para cada par (câmera, célula candidata), verifica-se se uma parede intercepta o segmento de reta entre os dois pontos; se houver, a célula permanece em *sombra* (ponto cego), mesmo dentro do alcance nominal da câmera.
- (B) **Campo de visão (FOV) limitado e direção fixa:** câmeras reais não giram 360° instantaneamente; são instaladas com direção fixa (N/S/L/O) e ângulo de abertura limitado (ex. 90°). Uma célula só é visível se estiver dentro do alcance máximo, dentro do cone de abertura, e com linha de visão livre (item A).

Essas variáveis aproximam a modelagem de um cenário real de projeto de CFTV (Circuito Fechado de TV), em contraste com a Galeria de Arte clássica, que assume visão em 360° sem obstáculos internos.

3. Solução Proposta

Dada a inviabilidade da solução exata, foi implementada uma heurística de duas fases — construção gulosa seguida de busca local — ambas em tempo polinomial.

3.1. Cálculo de visibilidade

Para cada câmera candidata (p, d) , o conjunto de células visíveis é calculado por *Ray-casting*, usando o algoritmo de Bresenham para traçar o segmento entre a câmera e cada célula candidata dentro do alcance R (Algoritmo 3.1). Uma célula é visível se estiver dentro do alcance, dentro do cone de FOV centrado na direção d , e se o segmento até ela não cruzar nenhuma parede.

[ht] **Entrada:** posição $p = (r, c)$, direção d , alcance R , FOV θ **Saída:** conjunto V de células visíveis $V \leftarrow \emptyset$ cada célula livre q tal que $\text{dist}(p, q) \leq R \wedge \leftarrow$ ângulo de q em relação a $p \wedge$ dentro do cone de FOV centrado em d e segmento $p \rightarrow q$ livre (Bresenham) $V \leftarrow V \cup \{q\}$ V

3.2. Fase 1 — Heurística gulosa (Greedy Set Cover)

A cada iteração, todas as combinações de posição e direção disponíveis são avaliadas, e escolhe-se a de maior *ganho marginal* (células ainda não cobertas que passariam a ser cobertas). A câmera escolhida é adicionada à solução, repetindo-se até atingir K câmeras, a cobertura desejada, ou nenhum ganho adicional ser possível. Esta é a heurística clássica para Set Cover, com fator de aproximação $\ln(n)$ em relação ao ótimo [3], e ilustra bem o princípio geral dos algoritmos gulosos — a cada escolha, tomar a decisão ótima naquele momento, na expectativa de que o conjunto de escolhas locais leve a uma solução global satisfatória [4].

3.3. Fase 2 — Busca local

Cada câmera posicionada é removida temporariamente, e busca-se a melhor posição/direção alternativa considerando fixa a cobertura das demais. Se a nova posição aumenta a cobertura total, a câmera é realocada. O processo repete-se até convergência (nenhuma câmera muda de posição) ou um número máximo de iterações, escapando de mínimos locais da escolha gulosa sequencial.

3.4. Complexidade e implementação

Sejam N o número de células livres, R o alcance e K o número de câmeras. A fase gulosa avalia, em cada uma das K iterações, N posições e 4 direções, cada uma percorrendo uma vizinhança $O(R^2)$ — complexidade $O(K \cdot N \cdot R^2)$, polinomial, contra o $O(2^{4N})$ da busca exaustiva. O algoritmo foi implementado em C (Código 1 mostra o núcleo do raycasting); o código-fonte completo é material suplementar deste trabalho.

Listing 1. Núcleo do raycasting (Bresenham) para linha de visão.

```
1 int linha_de_visao_livre(Mapa *m, int r0, int c0, int r1, int c1
  ) {
2     int dr = abs(r1-r0), dc = abs(c1-c0);
3     int sr = (r0<r1)?1:-1, sc = (c0<c1)?1:-1;
4     int err = dr-dc, r = r0, c = c0;
5     while (r != r1 || c != c1) {
6         int e2 = 2*err;
7         if (e2 > -dc) { err -= dc; r += sr; }
8         if (e2 <  dr) { err += dr; c += sc; }
9         if (r==r1 && c==c1) break;
```

```

10         if (m->grid[r][c] == 1) return 0; /* obstaculo bloqueia
        */
11     }
12     return 1; /* linha de visao livre */
13 }

```

4. Testes Realizados e Resultados

O algoritmo foi avaliado em dois cenários sintéticos, representando ambientes candidatos à instalação de CFTV.

4.1. Cenário 1 — Galpão com pilastras

Um mapa de 15×20 células (148 livres) simula um galpão com corredores e pilastras internas. Parâmetros: $K = 12$ câmeras, FOV 90, alcance $R = 8$. A fase gulosa atingiu 74,3% de cobertura (110 de 148 células), com ganho marginal decrescente (14 células na primeira câmera, 5 na última). A busca local encontrou, para a câmera 4, uma posição alternativa — (11, 8) Leste, em vez da original (11, 18) Oeste — que cobre o mesmo número de células. Esse empate ocorre porque a área originalmente exclusiva da câmera 4 passou a se sobrepor, parcialmente, com regiões já cobertas pelas câmeras adicionadas posteriormente pelo guloso; como não houve posição estritamente melhor, a realocação manteve o percentual final inalterado, indicando que o guloso já alcançara um bom mínimo local. Os 38 pontos cegos remanescentes concentram-se atrás de pilastras, exigindo mais câmeras ou maior FOV para eliminação total (Figura 1).

```

Mapa apos busca local (resultado final):

01234567890123456789
0 #####
1 #v??v?#>.....#v???#
2 #.##..#?#####.?#.#?#
3 #.##...?#.?#.?#.#
4 #.#####.#.#.#.#
5 #...#.....<?#..?#
6 #####.#?#####.#.#.#
7 #?????#...<#...?#
8 #?###.#?###.##^###.#
9 #???#?#?#..???#??#
10 #####.#.#####.#^#
11 #>.....>.....??#
12 #?#####.#####.#
13 #?.....<.....<#
14 #####

Legenda: # parede | . coberto | ? ponto cego (nao coberto) | ^ > v < camera (direcao)

===== RELATORIO FINAL =====
Dimensao do mapa: 15 x 20
Celulas livres (area util): 148
Cameras utilizadas: 12
FOV por camera: 90 graus | Alcance: 8 celulas
Celulas cobertas: 110
Pontos cegos remanescentes: 38
Cobertura total: 74.32%

```

Figura 1. Mapa final do Cenário 1 (galpão) após as duas fases da heurística.

4.2. Cenário 2 — Agência bancária (mapa sintético)

Um segundo mapa, de 7×12 células (35 livres), foi construído sinteticamente para representar uma pequena agência bancária, com salas segregadas por paredes internas. Com $K = 6$ câmeras, FOV 90, alcance $R = 6$, o algoritmo atingiu 94,3% de cobertura, restando 2 células em ponto cego, em um canto de difícil acesso visual (Figura 2).

```

Mapa apos busca local (resultado final):

012345678901
0 #####
1 #v..?#>...#
2 #.#..#?##.#
3 #.#...<#>..#
4 #.###.#.###
5 #....><..#
6 #####

Legenda: # parede | . coberto | ? ponto cego (nao coberto) | ^ > v < camera (direcao)

===== RELATORIO FINAL =====
Dimensao do mapa: 7 x 12
Celulas livres (area util): 35
Cameras utilizadas: 6
FOV por camera: 90 graus | Alcance: 6 celulas
Celulas cobertas: 33
Pontos cegos remanescentes: 2
Cobertura total: 94,29%

```

Figura 2. Mapa final do Cenário 2 (agência bancária) após as duas fases da heurística.

4.3. Discussão

Os resultados mostram sensibilidade à topologia: ambientes labirínticos com múltiplas pilastras (Cenário 1) exigem mais câmeras ou maior FOV, enquanto ambientes mais abertos (Cenário 2) permitem cobertura quase total com poucas câmeras. Em ambos os casos, o tempo de execução foi da ordem de milissegundos, validando empiricamente a complexidade polinomial discutida na Seção 3. A busca local contribuiu principalmente na redistribuição de câmeras, com ganho percentual discreto — esperado, já que o guloso já produz soluções próximas de seu limite teórico de aproximação.

5. Conclusão

Este trabalho apresentou uma heurística para posicionamento de câmeras de vigilância, modelado como variação do Problema da Galeria de Arte / Cobertura de Conjuntos, ambos NP-difíceis, incorporando linha de visão sensível a obstáculos e FOV limitado com direção fixa — restrições que aproximam a solução de projetos reais de CFTV.

A heurística de duas fases, implementada em C, produziu soluções de boa qualidade (74% a 94% de cobertura, conforme a topologia) em tempo polinomial, evidenciando a diferença de escalabilidade em relação à busca exaustiva, cuja complexidade $O(2^{4N})$ inviabiliza sua aplicação mesmo em ambientes de tamanho moderado. Como trabalhos futuros, destacam-se ângulos de direção contínuos (não restritos a N/S/L/O), custos diferenciados por posição de instalação, e comparação com meta-heurísticas populacionais (algoritmos genéticos, colônia de formigas) em ambientes de maior escala.

Referências

- [1] Vídeo de referência sobre posicionamento de câmeras de vigilância. Disponível em: https://www.youtube.com/watch?v=WMvlHk_6guY. Acesso em: jul. 2026.
- [2] GEEKSFORGEEEKS. *Art Gallery Problem*. Disponível em: <https://www.geeksforgeeks.org/dsa/art-gallery-problem/>. Acesso em: jul. 2026.
- [3] GEEKSFORGEEEKS. *Greedy Approximate Algorithm for Set Cover Problem*. Disponível em: <https://www.geeksforgeeks.org/dsa/greedy-approximate-algorithm-for-set-cover-problem/>. Acesso em: jul. 2026.
- [4] PINTO, L. I. *Tratabilidade de Problemas Computacionais e Meta-Heurísticas*. Slides da aula da disciplina de Complexidade de Algoritmos. UDESC.